90-DAY EXECUTION GUIDE

90 DAYS BEFORE 2027

My Practical Data Engineering Switch Plan

A focused after-work roadmap for SQL, Python, one production-style project, early applications, and interview practice.

PRANJAL PATIDAR
Senior Data Engineer - Data with Pranjal



FREE WORKBOOK - PERSONAL USE - VERSION 1.0

START HERE

Use this guide, not another saved roadmap

This plan is for someone with a full-time job who already has some exposure to SQL or programming and can protect about 16 focused hours per week. Adjust the stack to your target roles, but do not keep changing the plan every seven days.

OCTOBER	NOVEMBER	DECEMBER
Make SQL, Python, Git, and your first report usable.	Make one pipeline survive real problems - and begin applying.	Practise interviews, fix weak explanations, and improve from feedback.

The honest promise

What 90 days can do
Give you a credible project, a repeatable study system, interview practice, and a real application pipeline. It cannot guarantee a job offer or replace the experience requirements of every company.

Rules that protect your time

- [ ] Choose one target role and one primary stack before Day 1.
- [ ] Build one project deeply enough to explain its failures, not five shallow repos.
- [ ] End every study topic by producing something without the solution open.
- [ ] Start applying between Days 30 and 45; do not wait for perfection.
- [ ] Use AI to ask better questions, not to hide gaps in your understanding.
- [ ] If life interrupts the plan, continue from the next task. Do not restart from Day 1.

BEFORE DAY 1

Analyse 15 roles before choosing your stack

The market does not hire a generic roadmap. Collect 15 relevant job descriptions, tally repeated requirements, and use that evidence to decide what deserves your next 200 hours.

#	COMPANY / ROLE	SQL + PYTHON	CLOUD	SPARK / STREAM	ORCHESTRATION + NOTES
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

Turn the tally into one sentence

> **Target statement**
> For the next 90 days, I am preparing for **[target role]** positions that repeatedly require **[SQL]**, **[Python]**, **[cloud]**, and **[one additional platform skill]**. Everything else goes on my Not Now list.

- [ ] Select roles you would genuinely apply to, not only famous-company postings.
- [ ] Separate must-have requirements from optional preferences.
- [ ] Add Spark only if it appears often enough in your target sample.
- [ ] Choose one cloud based on the sample; do not learn AWS, Azure, and GCP together.

THE MAP

Your 90-day plan on one page

OCTOBER
DAYS 1-30

Build usable foundations
SQL correctness
Python ingestion
Git history
Verified first report
Resume draft

DAY 30: working local pipeline + checked revenue

NOVEMBER
DAYS 31-60

Build proof and apply
Idempotency
Late data + corrections
Airflow recovery
Optional PySpark
5 focused applications/week

DAY 60: scheduled pipeline + one recovered failure

DECEMBER
DAYS 61-90

Perform in interviews
Two mocks/week
Five-minute project story
Pipeline design questions
Application feedback loop
Final demonstration

DAY 90: explainable portfolio + active applications

Suggested allocation for about 200 focused hours

WORKSTREAM	HOURS	WHAT THE TIME MUST PRODUCE
SQL	35	Queries, checks, grain decisions, and a verified revenue result
Python	25	Repeatable ingestion, cleaning, logging, and tests
Core project	65	End-to-end pipeline, failure handling, README, and demo
Airflow / cloud / optional Spark	25	Scheduling, recovery, and one role-specific extension
Interview practice	30	Mocks, follow-ups, project story, and design explanations
Resume + applications	20	Role targeting, tailored applications, and feedback tracking

YOUR WEEK

A schedule that can survive a full-time job

The plan uses output blocks, not vague study hours. Stop each session with the next action written down so you can restart quickly after work.

WEEKDAY - 2 HOURS
10 min plan
60 min learn or solve
40 min build
10 min explain + log

WEEKEND - 3 HOURS
15 min plan
120 min build
30 min validate
15 min document

EXHAUSTED DAY - 5 MIN
Fix one query
Improve one resume line
Explain one design decision aloud
Choose tomorrow's first task

Default weekly rhythm

DAY	PRIMARY BLOCK	OUTPUT
Monday	Learn one concept	Notes plus one example written from memory
Tuesday	Solve	One working query or function with checks
Wednesday	Build	One project increment committed to Git
Thursday	Break and recover	A failed case reproduced and understood
Friday	Explain	Two-minute explanation without notes
Saturday	Deep build	Integrated project feature or recovery test
Sunday	Validate and review	Evidence folder, weekly score, next-week task

Recovery rule
Missing two days does not invalidate the roadmap. Drop optional work, keep the week's core deliverable, and continue. Never spend a weekend rebuilding the plan instead of rebuilding the project.

THE CORE PROJECT

Build one daily revenue pipeline you can defend

The project is intentionally ordinary. Revenue pipelines force you to reason about grain, corrections, duplicates, late events, scheduling, validation, and business trust - the decisions that make a project interview-worthy.



orders + items + payments + refunds

daily_revenue.csv + checks + run log

Minimum datasets

TABLE	ONE ROW MEANS	IMPORTANT FIELDS
orders	One order	order_id, customer_id, order_ts, status, updated_at
order_items	One item in one order	order_id, item_id, quantity, unit_price
payments	One payment attempt or capture	payment_id, order_id, amount, status, paid_at
refunds	One refund event	refund_id, order_id, amount, refunded_at

Business result

Daily revenue mart
Produce one row per business date with gross paid amount, refunded amount, and net revenue. Keep a control total and enough run metadata to prove which inputs created the published result.

Repository evidence to keep

- [ ] README with the business question and architecture
- [ ] sample_data/ with small, explainable inputs
- [ ] sql/ and src/ separated from generated outputs
- [ ] tests/ for correctness and rerun behaviour
- [ ] run_logs/ with one successful and one recovered failure
- [ ] docs/ with your grain decisions and trade-offs

PROVE IT WORKS

Six tests that turn a demo into engineering evidence

Do not call the pipeline complete because it ran once. Reproduce the failure, write the expected result, run the fix, and preserve the evidence.

TEST	BREAK IT	PASS CONDITION
Duplicate delivery	Run the same input file twice	Second run does not duplicate rows or revenue
Child-table fan-out	Use two payments and two refunds for one order	Pre-aggregation prevents multiplied amounts
Order correction	Change a prior order from paid to cancelled	Published revenue reflects the latest valid state
Late refund	Deliver a refund seven days after payment	Prior business date is recomputed or restated by policy
Task failure	Fail validation or publishing deliberately	Retry does not corrupt or double-publish the result
Empty input	Deliver a valid but empty file	Pipeline distinguishes no data from a silent extraction failure

For every test, save four things

1. INPUT
The exact rows or condition that creates the edge case.

2. EXPECTATION
The row count, control total, or state you expect.

3. EVIDENCE
The actual output, log line, and explanation of the fix.

OPTIONAL PRACTICE CHECKPOINT

Can you stop a payments-and-refunds join from multiplying revenue?

The free TDF scenario uses the same grain problem as this project. Solve it before reading the model explanation.

TRY THE FREE SCENARIO

OCTOBER - WEEK 1 - DAYS 1-7

Set up the project and make SQL visible

By Sunday, another person should be able to clone the repository, load the sample data, and run your first checked query.

Your work this week

[ ] Create the repository and a README with the business question.

[ ] Create small orders, order_items, payments, and refunds files.

[ ] Practise filtering, grouping, joins, null handling, and basic CTEs.

[ ] Write five queries without keeping the solution open.

[ ] Commit small changes with messages that explain the outcome.

SHIP	PROVE	BUSY-WEEK VERSION
A runnable repository with sample data, five SQL queries, and setup instructions.	- State the grain of every input table - Predict row counts before joining - Reconcile one query manually	Load two tables and write one correct revenue query. Preserve the next task for Monday.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

OCTOBER - WEEK 2 - DAYS 8-14

Learn the SQL that protects business numbers

Move beyond syntax: make joins, window functions, and aggregates produce a result you can defend to Finance.

Your work this week

[ ] Write the daily paid amount at the correct business-date grain.

[ ] Aggregate payments and refunds before joining them to orders.

[ ] Use a window function for the latest order state when updates repeat.

[ ] Create row-count and control-total checks before and after joins.

[ ] Add at least three deliberately tricky test inputs.

SHIP	PROVE	BUSY-WEEK VERSION
A verified SQL revenue transformation with expected outputs and failure cases.	- Explain why a query can run and still be wrong - Show the fan-out failure - Show the corrected control total	Finish the grain definition and one join-multiplication test. Skip extra exercises.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

OCTOBER - WEEK 3 - DAYS 15-21

Use Python to ingest and clean data safely

Build a small Python entry point that reads raw files, normalises values, rejects bad rows, and produces a repeatable staged output.

Your work this week

[ ] Parse dates, decimals, identifiers, and status values explicitly.

[ ] Separate valid rows from rejected rows with a reason code.

[ ] Add logging for files read, rows accepted, rows rejected, and output written.

[ ] Write tests for missing fields, invalid amounts, and duplicate IDs.

[ ] Make the script rerunnable from one documented command.

SHIP	PROVE	BUSY-WEEK VERSION
A tested Python ingestion step with staged data and a rejection report.	- Explain one rejected row - Run twice with the same output - Show a failing test before the fix	Read one file, validate required fields, and write accepted and rejected outputs.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

OCTOBER - WEEK 4 - DAYS 22-30

Connect ingestion, SQL, and a checked report

Finish October with a working local pipeline and a first resume draft - not with another course certificate.

Your work this week

[ ] Connect the Python staging output to your SQL transformation.

[ ] Generate a daily revenue report and compare it to a manual control total.

[ ] Document setup, architecture, grain, assumptions, and known limitations.

[ ] Record a three-minute private walkthrough for yourself.

[ ] Write the first honest project bullet for your resume.

SHIP	PROVE	BUSY-WEEK VERSION
Input -> cleaned data -> verified revenue report, with README and demo evidence.	- A fresh setup works - Numbers reconcile - You can explain the pipeline without notes	Publish one checked report and update the README. Defer polish and optional tools.

Decision rule
You may begin targeted applications as soon as this core path works. Do not wait for November if you can explain the project honestly.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

NOVEMBER - WEEK 5 - DAYS 31-38

Make reruns safe and start applying

The project now becomes evidence of engineering judgment. The same input should not silently create a different business result.

Your work this week

[ ] Introduce a batch_id or source-file identity and record processed inputs.

[ ] Run the same delivery twice and prove that output remains correct.

[ ] Choose an idempotent write strategy and document its trade-off.

[ ] Tailor your resume to two role families, not fifteen unrelated roles.

[ ] Submit five carefully selected applications and track each result.

SHIP	PROVE	BUSY-WEEK VERSION
An idempotent rerun demonstration plus the first five tracked applications.	- Show duplicate protection - Explain overwrite vs merge - Connect one resume bullet to the role	Implement the processed-file check and submit two high-match applications.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

NOVEMBER - WEEK 6 - DAYS 39-45

Handle corrections, cancellations, and late refunds

A daily pipeline is incomplete until it can explain why yesterday's certified number changed today.

Your work this week

[ ] Apply updates using a stable business key and updated_at ordering.

[ ] Deliver a cancellation after the original paid order.

[ ] Deliver a refund seven days late and define the restatement policy.

[ ] Keep both raw history and the current trusted state.

[ ] Write a reconciliation query between payment totals and the revenue mart.

SHIP	PROVE	BUSY-WEEK VERSION
A late-change test with documented business policy and reconciled output.	- Explain event time vs processing time - Show the before/after result - Defend your restatement choice	Implement one late-refund case and document the expected result.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

NOVEMBER - WEEK 7 - DAYS 46-53

Schedule the work and recover from failure

Use an orchestrator such as Airflow to make dependencies, retries, checks, and recovery visible.

Your work this week

[ ] Create tasks for load, validate, transform, publish, and reconcile.

[ ] Add bounded retries only for genuinely transient work.

[ ] Fail validation deliberately and verify that publishing does not run.

[ ] Retry the failed step and prove that revenue is not duplicated.

[ ] Record one incident note: impact, evidence, cause, fix, prevention.

SHIP	PROVE	BUSY-WEEK VERSION
A scheduled pipeline with one deliberately failed and safely recovered run.	- Show task dependencies - Explain retry safety - Show the recovered control total	Create a three-task DAG: validate -> transform -> publish. Fail validation once.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

NOVEMBER - WEEK 8 - DAYS 54-60

Add PySpark only when your target roles justify it

Do not rebuild the entire project in Spark for appearances. Rebuild one transformation that helps you discuss partitions, joins, and distributed execution.

Your work this week

[ ] Check how often Spark appears in the 15-role sample.

[ ] If justified, rebuild the revenue transformation with DataFrames.

[ ] Partition by a useful business date and inspect the physical plan.

[ ] Avoid collect-to-driver and unnecessary Python UDF patterns.

[ ] If Spark is not required, use this week for tests, cloud deployment, or stronger applications.

SHIP	PROVE	BUSY-WEEK VERSION
One role-relevant extension with a written reason for including it.	- Explain partition choice - Compare local SQL and Spark outputs - Name one scale boundary	Do not add Spark. Strengthen the current pipeline and submit five applications.

Decision rule
Add Spark only when it appears in at least one-third of your selected roles or is a clear requirement for the roles you want most.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

DECEMBER - WEEKS 9-10 - DAYS 61-74

Turn the project into a five-minute interview story

The project only helps if you can explain the business problem, one decision, one failure, and the evidence that proves your fix.

Your work this week

[ ] Write a five-part story: context, requirement, decision, failure, result.

[ ] Run one SQL/Python mock and one pipeline-design mock each week.

[ ] Ask the interviewer to interrupt with follow-up questions.

[ ] Record your explanation and remove vague claims or tool lists.

[ ] Update the README where your verbal explanation exposes missing evidence.

SHIP	PROVE	BUSY-WEEK VERSION
A five-minute project explanation plus four completed mock sessions.	- Answer without slides - Quantify only what you measured - Explain why, not only what	Record one five-minute explanation and answer three follow-up questions.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

DECEMBER - WEEKS 11-12 - DAYS 75-84

Use application and mock feedback to choose fixes

Do not respond to every rejection by learning a new tool. Diagnose the stage that is failing and change the smallest relevant thing.

Your work this week

[ ] No recruiter response: review role match, resume evidence, and application quality.

[ ] Screening failures: practise concise project and experience stories.

[ ] Technical failures: replay the exact concept and add one project test.

[ ] Design failures: clarify requirements, trade-offs, scale, and reliability.

[ ] Continue five targeted applications per week and record outcomes.

SHIP	PROVE	BUSY-WEEK VERSION
A feedback log where every repeated failure maps to one specific improvement.	- Name the failed stage - Show the change you made - Retest the same weakness	Review the last five applications and fix one repeated problem only.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

DECEMBER - WEEK 13 - DAYS 85-90

Finish with proof, not a new learning list

Enter January with a project you can run, explain, and improve - plus an application process already in motion.

Your work this week

[ ] Run the project from a clean setup and fix broken instructions.

[ ] Complete the six failure tests and save their evidence.

[ ] Deliver one final five-minute explanation and one design mock.

[ ] Review application conversion by stage, not only total applications.

[ ] Write the next 30-day plan based on real feedback.

SHIP	PROVE	BUSY-WEEK VERSION
A final demo, verified repository, active application tracker, and evidence-based January plan.	- Fresh run succeeds - Failure recovery is visible - You know the next constraint	Run the project, record the explanation, and choose one January priority.

Weekly review

[ ] What did I finish that another person can inspect?

[ ] What result did I verify instead of assuming?

[ ] What confused me when I explained it without notes?

[ ] What is the single next task for Monday?

APPLICATION TRACK

Apply before you feel finished

Begin between Days 30 and 45. Five relevant applications per week are more useful than fifty random clicks because each result can teach you something about positioning, role fit, or interview readiness.

COMPANY	ROLE	MATCH / 5	APPLIED	STAGE	NEXT ACTION

Weekly application diagnosis

SIGNAL	LIKELY QUESTION	NEXT TEST
No responses	Is the role match or evidence weak?	Tailor five applications; review the top third of the resume
Recruiter calls, no technical rounds	Is the story unclear or experience mispositioned?	Practise a 60-second transition story
Technical rounds, no progress	Which concept repeats?	Replay one failure and add one project test
Final rounds, no offer	Which trade-off or communication gap appears?	Run one targeted mock and request specific feedback

Resume bullet template
Built a **[daily / scheduled]** revenue pipeline using **[tools actually used]**; handled **[duplicate inputs, late corrections, or recovery]**; verified correctness using **[control totals, tests, and reconciliation]**. Replace brackets only with facts you can demonstrate.

INTERVIEW TRACK

Practise the questions your project creates

A strong project explanation invites deeper questions. That is good. Use the questions below to expose weak reasoning before an interviewer does.

1. What does one row mean in every table?	**2.** How can two valid child tables multiply revenue after a join?
3. What happens when the same source file arrives twice?	**4.** How do you distinguish a late event from a duplicate event?
5. Would a late refund rewrite history or create an adjustment? Why?	**6.** Where is the pipeline state stored, and what happens if it is lost?
7. Which retry is safe, and which retry can duplicate output?	**8.** How do you know yesterday's number is complete?
9. What would change at 100x data volume?	**10.** Why did you choose SQL, Python, Airflow, or Spark for each step?
11. What trade-off did you accept to finish in 90 days?	**12.** What would you improve next if this became a real production system?

Five-minute explanation structure

- [ ] 45 seconds - business question and required output
- [ ] 60 seconds - inputs, grain, and architecture
- [ ] 75 seconds - one decision and why you made it
- [ ] 75 seconds - one failure, evidence, fix, and prevention
- [ ] 45 seconds - limitations and what you would improve next

USE AI WITHOUT OUTSOURCING THE THINKING

Prompts that make AI challenge you

Never paste employer data, customer data, secrets, or proprietary code into public AI tools. Use synthetic project inputs and ask the model to reveal questions before solutions.

SOCRATIC DEBUGGER

Here is my SQL query and expected result. Ask one question at a time that helps me find the bug. Do not show the corrected query unless I ask.

GRAIN REVIEWER

Review only the table grains and join cardinalities in this design. Identify where facts could multiply, but make me propose the fix first.

FAILURE INJECTOR

Give me five realistic failures for this pipeline. For each, state the observable evidence but not the root cause.

MOCK INTERVIEWER

Interview me about this project for 20 minutes. Interrupt vague answers and ask follow-ups about retries, late data, idempotency, and validation.

TRADE-OFF CHALLENGER

Challenge one design choice at a time. Ask what I gain, what I lose, and what evidence would make me change the decision.

RESUME CRITIC

Flag claims in this project bullet that sound vague, inflated, or impossible to verify. Do not rewrite it until I supply evidence.

TEST DESIGNER

Given this expected business result, ask me for edge cases. After I answer, point out missing correctness tests.

WEEKLY COACH

Using this week's deliverable and evidence, ask what I shipped, verified, failed, and will do next. Reject activity-only answers.

DAY 90

The evidence you should carry into January

You do not need every box to be perfect. You do need enough evidence to know what you can demonstrate, what still needs work, and what feedback should shape the next month.

[ ] I can state the grain of every project table.	[ ] My SQL result reconciles to a control total.	[ ] My Python ingestion separates valid and rejected rows.	[ ] A duplicate input does not duplicate revenue.	[ ] A late refund or correction produces an intentional result.
[ ] A failed step can be retried without corrupting output.	[ ] My repository can be run from its README.	[ ] I can explain one production failure with evidence.	[ ] I have delivered at least four mock interviews.	[ ] I have an active, measured application pipeline.

Optional resources from Pranjal

PRACTISE THE DECISIONS

The Data Foundry

Browser-based SQL, Python, PySpark, Airflow, AWS, production scenarios, and system-design practice. Start with free content and use it to test what you can do without notes.

OPEN TDF PRACTICE

PAID - OPTIONAL

Interview Bundle Advanced

A structured reference collection covering SQL, Python, Spark, Airflow, AWS, data modelling, system design, production issues, scenario practice, behavioural prep, and updates through Pranjal's Topmate offering.

VIEW THE PDF BUNDLE

Trust note
This guide is complete on its own. You do not need to purchase anything to follow the 90-day plan. The links above are optional resources created by Pranjal for learners who want interactive practice or a deeper interview reference library. Links and product descriptions were verified on 29 September 2026.

Start tonight

Save five job descriptions. Block the first study session. Choose the first revenue question your project will answer.